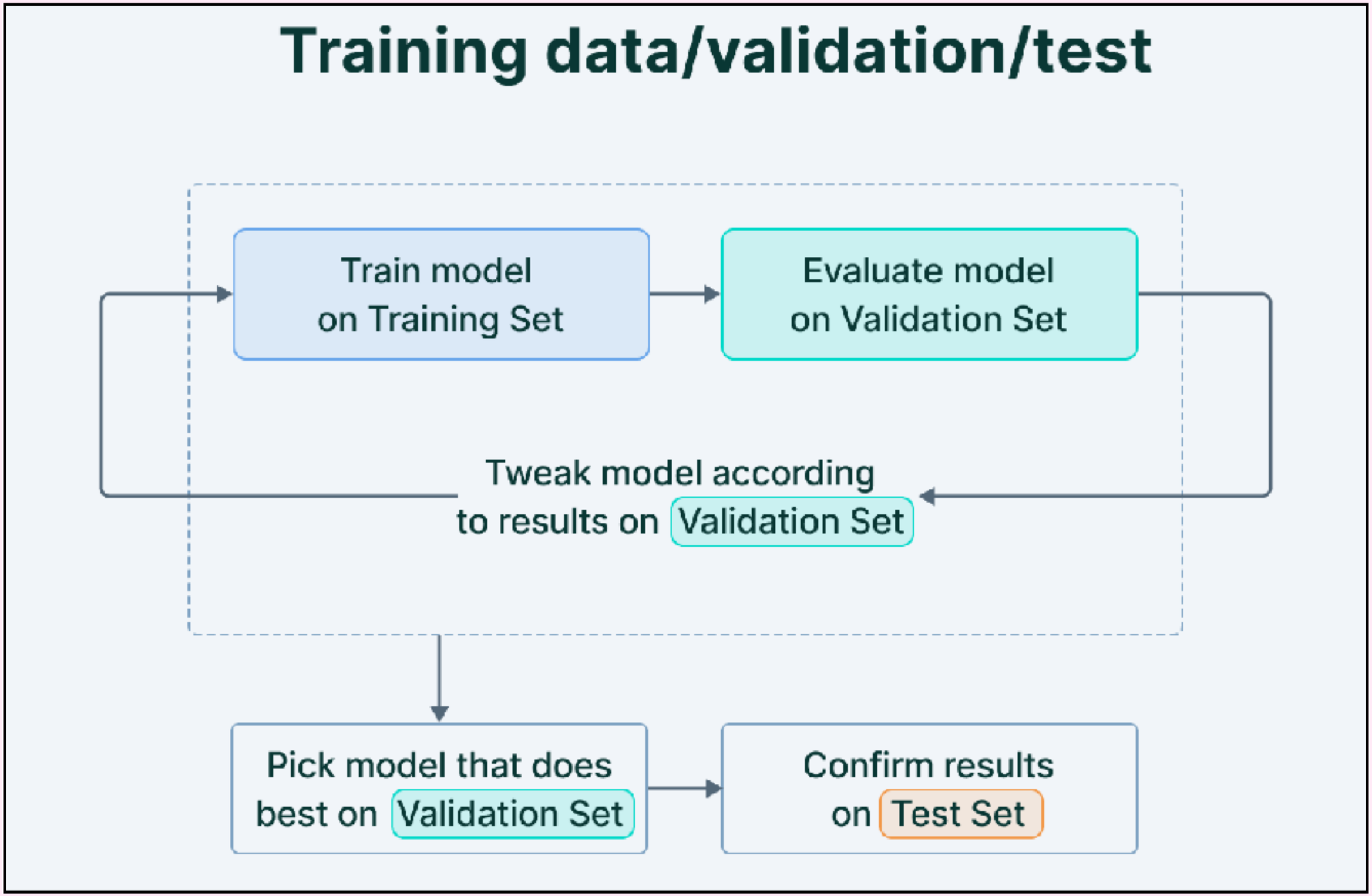
Artificial Neural Network

Why do they not work?
How can we evaluate training and model performance?

Dr Patricia Ryser-Welch, Oct 2026

How do we train a machine learning model?



Train model

Context: Classifying output of photovoltaic cells of solar panels: High or Low

- **Hyper-parameters:**
  - Neural architecture: input layer, interconnected layers of neutrons, and output layer
  - Activation functions: Relu, sigmoid
  - Optimisers and learning rates:
- **Algorithms used during training adjusting weights:**
  - Forward propagation and back propagation

Example code in Keras

Context: Classifying output of photovoltaic cells of solar panels: High or Low

Defining the model

```
model_e = tf.keras.Sequential([
    tf.keras.layers.Input(input_shape),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Layer (type)	Output Shape	Param #
dense_13 (Dense)	(None, 32)	448
dense_14 (Dense)	(None, 32)	1,056
dense_15 (Dense)	(None, 8)	264
dense_16 (Dense)	(None, 1)	9

Param Indicates the number of trainable parameters (weights and biases).

(13 x 32) + 32 biases =448 assume share of input is 13 items long

(32 x 32) + 32 biases = 1056
(32 x 8) + 8 bias =264
(8 x 1) + 1 bias = 9

https://www.mdpi.com/1996-1073/13/10/2570

Example code in Keras

Context: Classifying output of photovoltaic cells of solar panels: High or Low

Training the model

```
opt = optimizers.RMSprop(learning_rate=0.01)
model_e.compile(optimizer=opt,
                loss='binary_crossentropy',
                metrics=['accuracy'])
```

```
history = model_e.fit(train_data,
    validation_data=validation_data,
    epochs=40,
    verbose=0
        )
```

Accuracy and loss Roc-curves





https://www.mdpi.com/1996-1073/13/10/2570

How do we evaluate a model?

Evaluate: to judge, calculate, or determine the quality, value, importance, or condition of someone or something through careful consideration

Judgement: Made against some metrics

Careful consideration: Justify clearly against metrics within a problem domain or a context.

Can anyone remind me of any metrics we might use?

ANN : evaluation metrics

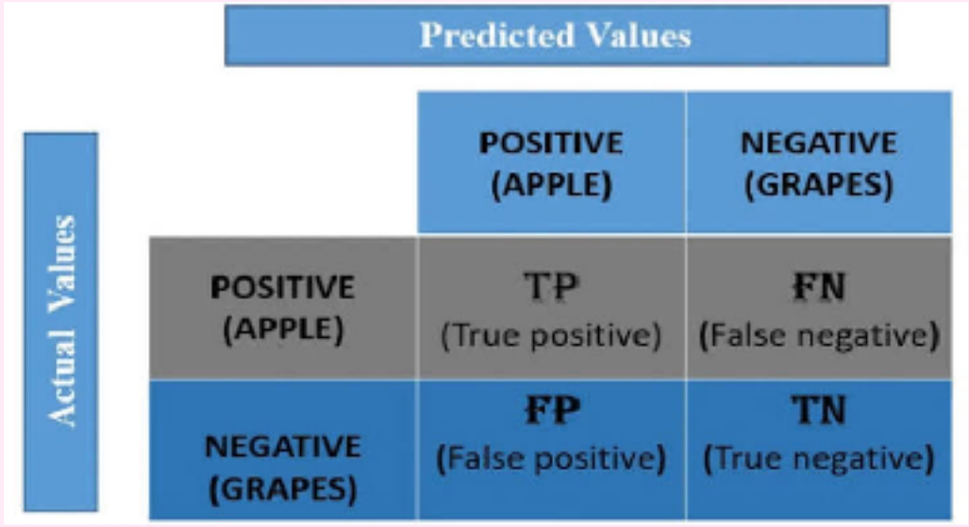
$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Recall (or TPR)} = \frac{\text{correctly classified actual positives}}{\text{all actual positives}} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{\text{correctly classified actual positives}}{\text{everything classified as positive}} = \frac{TP}{TP + FP}$$

Loss = Logarithm loss

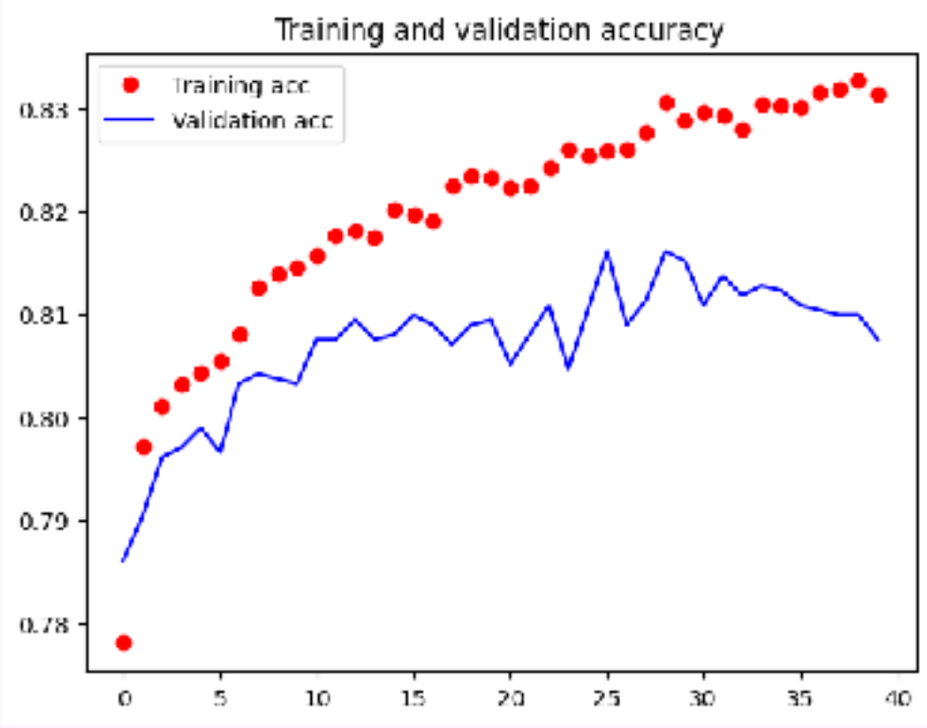
Roc- Curves and confusion matrix

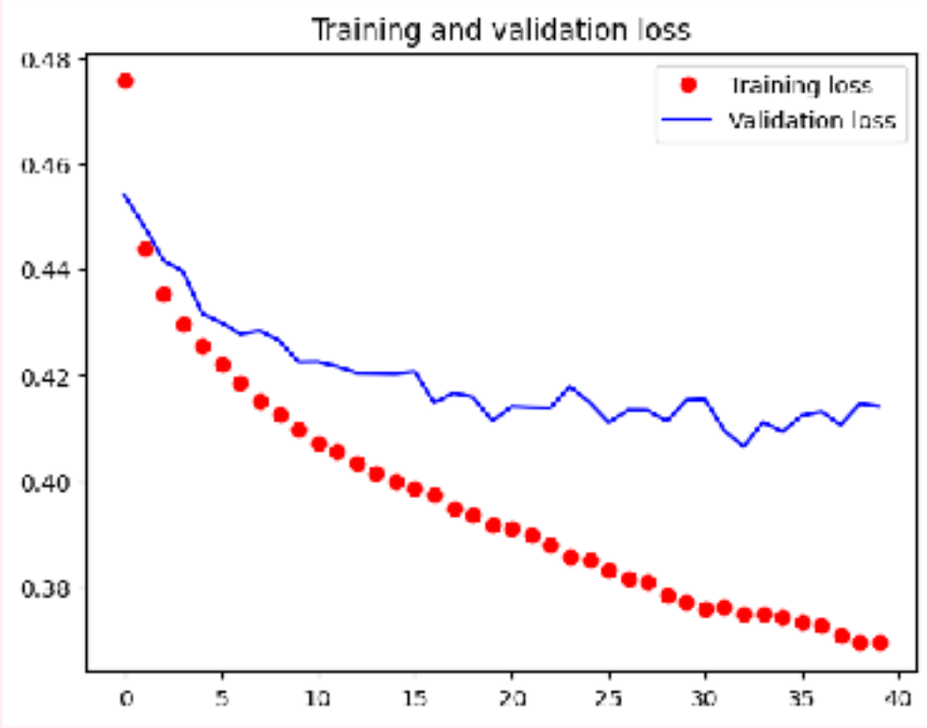


https://medium.com/@vishnuam/understanding-machine-learning-model-evaluation-metrics-58570f51a857

First challenge

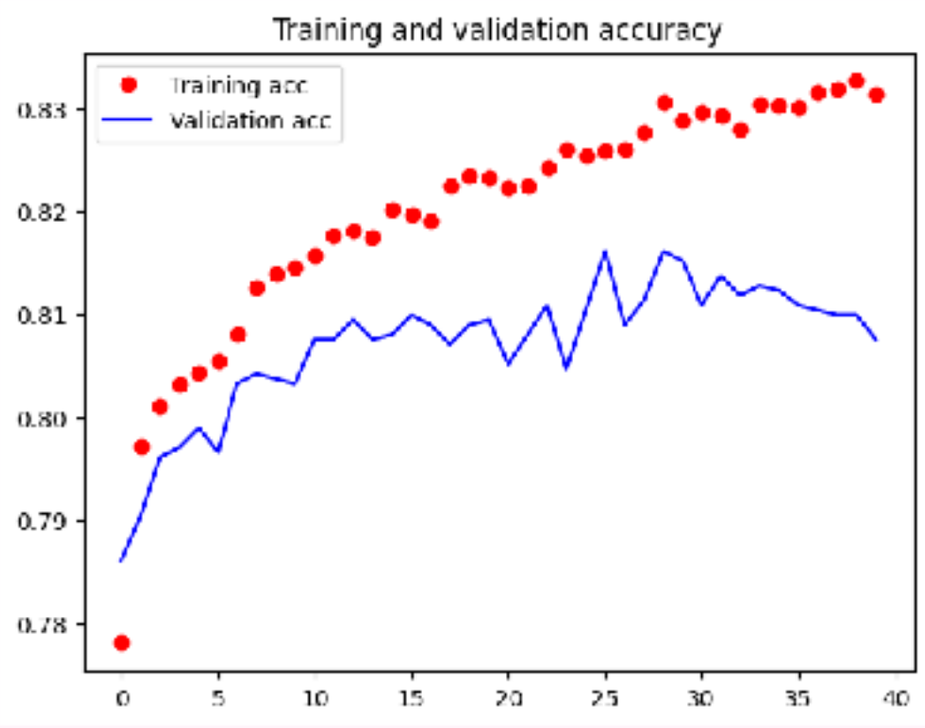
Evaluating the learning - what are your thoughts?

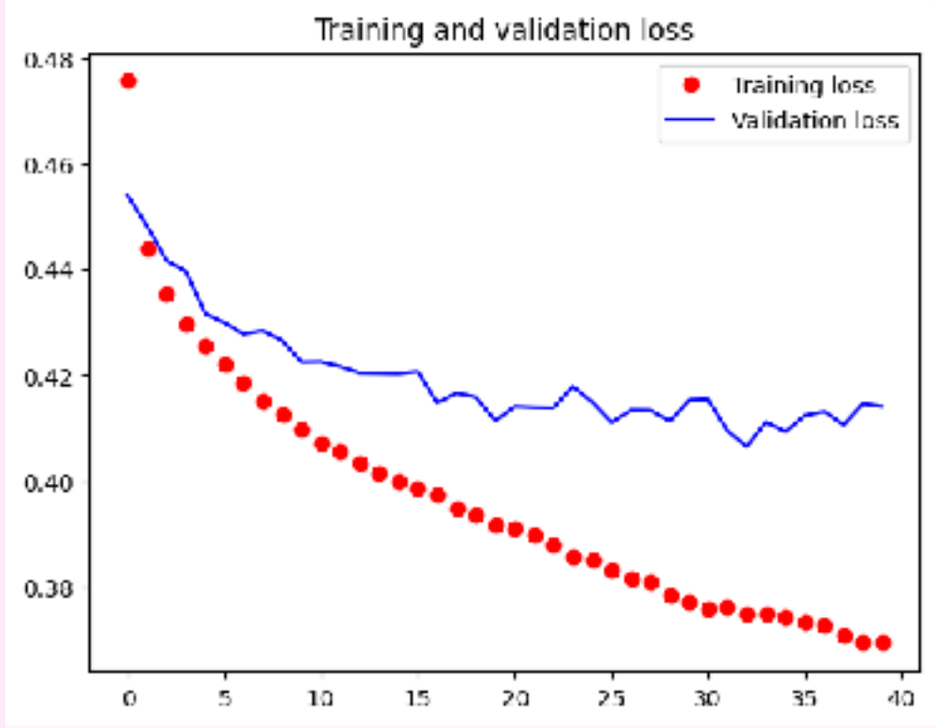




First challenge

Evaluating the learning - what are your thoughts?

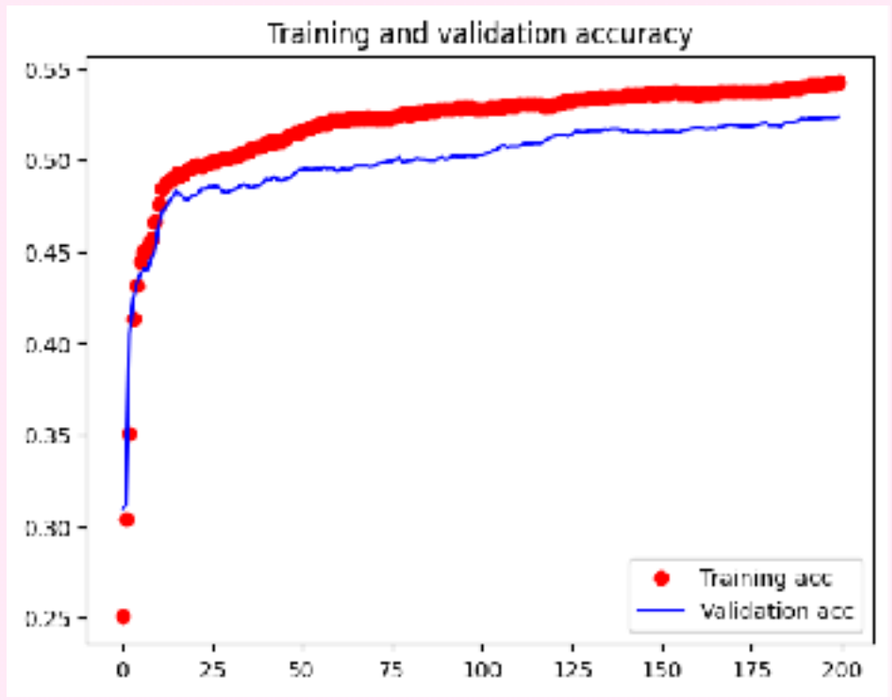


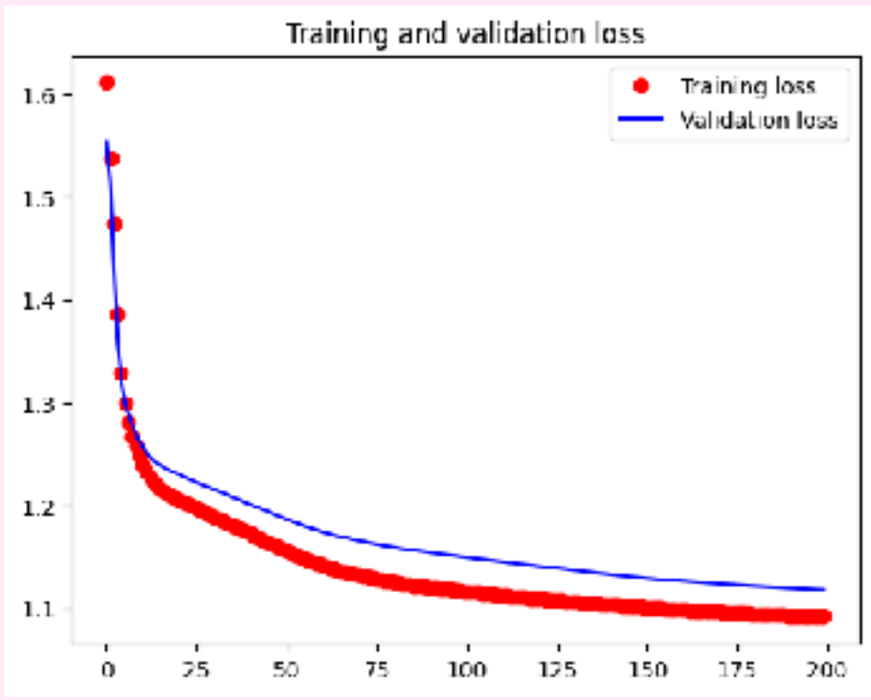


The classification of outputs from the solar cells has yet to be suitable - it is overfitting. The learning has adapted the weights to training datasets, but has failed to learn enough patterns that are also present in the validation dataset. The learning may have memorise some noise. Approximately after 25 epochs, the training accuracy continues to climb, while the validation falls. Similarly, the loss diverge for the training data, but drift for the validation dataset. Early stopping or augmenting both datasets might help reducing the drift.

Second challenge

Evaluating the learning - what are your thoughts?

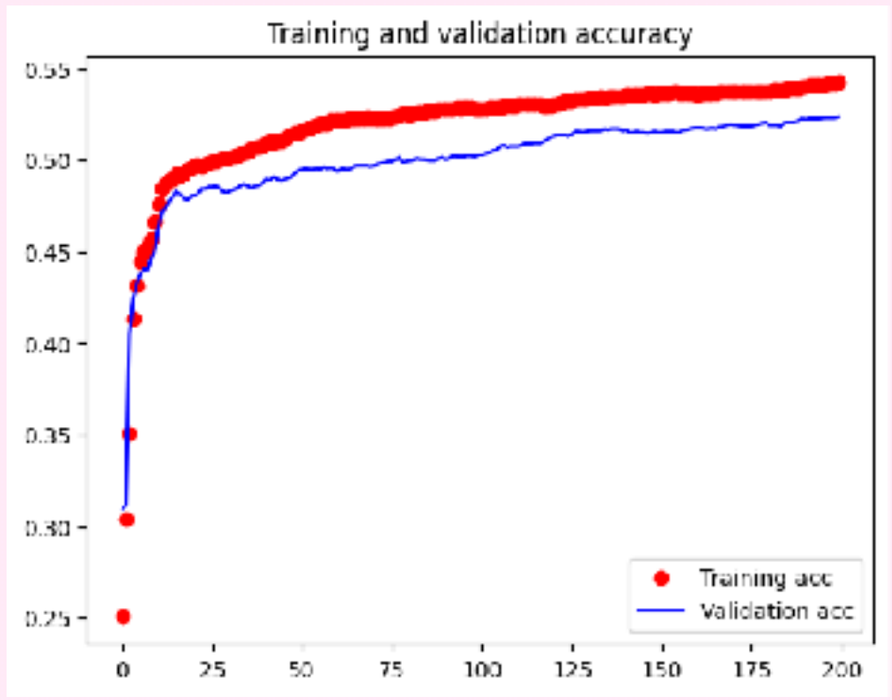


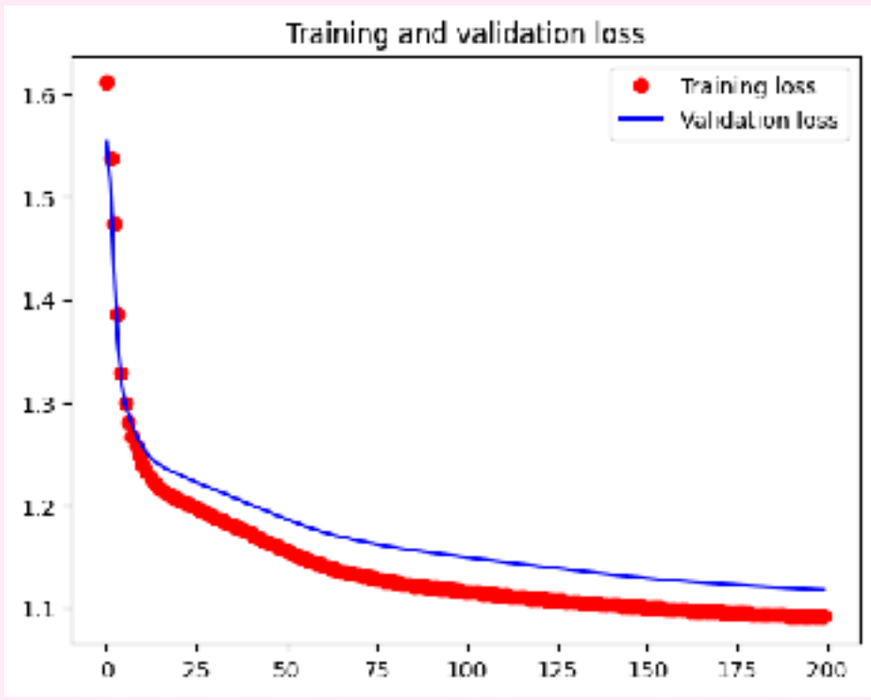


The classification of outputs from the solar cells is not suitable, but fitted well to the data . The learning has adapted the weights to training datasets, but has successfully learn enough patterns that are also present in the validation dataset. However the accuracy is too low for production.

Second challenge

Evaluating the learning - what are your thoughts?



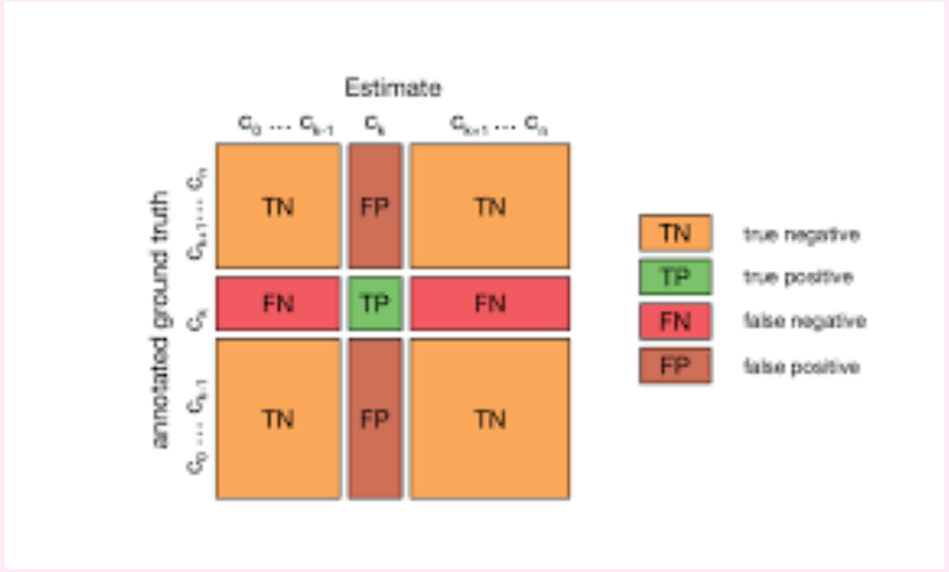


Third challenge - evaluating model

Context: Classifying output of photovoltaic cells of solar panels: bands 1,2,3,4,5

Confusion matrix

Actual labels \ Predicted labels	0	1	2	3	4
0	523	76	20	22	10
1	104	211	69	45	26
2	38	75	143	97	25
3	31	33	66	191	31
4	22	20	16	64	146



Evaluating models on unseen data

Context: Classifying output of photovoltaic cells of solar panels: bands 1,2,3,4,5
Which models would you chose?

Evaluating all the models on the test dataset:

```
model_a.evaluate(test_data, return_dict=True)
```

```
33/33 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5421 - loss: 1.0913
{'accuracy': 0.5292300581932068, 'loss': 1.0962635278701782}
```

```
model_b.evaluate(test_data, return_dict=True)
```

```
33/33 ━━━━━━━━━━━━━━━━━━━━ 0s 987us/step - accuracy: 0.5948 - loss: 1.1107
{'accuracy': 0.5874524712562561, 'loss': 1.129357933998108}
```

```
model_c1.evaluate(test_data, return_dict=True)
```

```
33/33 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5876 - loss: 1.0215
```

Why might ANN under-perform?

There are several common reasons why an Artificial Neural Network (ANN) might underperform or fail to yield good classification results on a dataset:

- **Unnormalized or Unscaled Data:** Neural networks rely heavily on inputs being on a similar scale (e.g., mean of zero and standard deviation of one). If features have widely different ranges or large values, the optimization and gradient updates become unstable. **Numerical values transformed between 0 and 1.**

- **Incorrect Learning Rate:** If the learning rate is set too high, training can diverge or bounce around; if it is set too low, the network may get stuck and fail to learn. **Have small learning attempts to discover most suitable learning rate.**

- **Insufficient Data:** ANNs generally require large amounts of labeled data to generalise effectively. With smaller datasets, simpler models often outperform neural networks. **Either change advanced statistical methodology or machine learning technique.**

- **Class Imbalance:** If certain classes appear much more frequently than others in the training set, the network can become biased toward predicting the majority classes while ignoring minority ones. **Try to remove class imbalances using real life and suitable data.**

- **Poor Architecture / Hyperparameters:** Having too few neurons or hidden layers can lead to underfitting (the model is too simple to capture patterns), while too many can lead to overfitting or vanishing/exploding gradients. **Trying over and over again - use published effective architecture.**

- **Bad Weight Initialisation or Activation Functions:** Using inappropriate activation functions (like standard ReLU suffering from dying neurons) or improper weight initialisation can prevent gradients from flowing properly during back propagation. **Explore the distribution of weight the lead to the best classification with the data available.**